

Projet d'Administration des Systèmes et des Réseaux II

Architecture de réseaux et commutation

Tony LENG, 20221861 & Paul VEROT, 20212888

Master I CNS-SR, 2026

Semestre 2



UNIVERSITÉ ÉVRY
PARIS-SACLAY



UNIVERSITÉ ÉVRY
PARIS-SACLAY

Département d'informatique
univ-evry.fr

Titre:

Projet d'Administration des Systèmes et des Réseaux II

Résumé:

--

Thème:

Architecture de réseaux et commutation

Groupe:

Master I CNS-SR

Participants:

Tony LENG, 20221861

Paul VEROT, 20212888

Email:

lengtony91@gmail.com

20221861@etud.univ-evry.fr

paul@paulverot.fr

20212888@etud.univ-evry.fr

Superviseur:

Mehdi Denou

Nombre de pages:

18

Dernier changement:

31-03-2026

Chapitres

1 Partage Réseau NFS	2
1.1 Résumé des contraintes et des demandes:	2
1.2 Mise en Place	3
1.2.1 Sur VMware	3
1.2.2 Sur les machines :	4
1.2.3 Explication des Commandes	6
1.3 Configuration des Adresses IP sur les Machines	6
1.3.1 Machine 1	6
1.3.2 Machine 2	7
1.3.3 Serveur NFS	7
1.3.3.1 Explication des paramètres du Bond	8
1.4 Configuration du Pare-feu	9
1.4.1 Sur Switch-3	9
1.4.1.1 Explication des règles de pare-feu	9
1.5 Configuration du Partage NFS	10
1.5.1 Sur le Serveur NFS	10
1.5.1.1 Test du transfert NFS	10
1.5.1.2 Test d'accès restreint	10
2 Redondance de liens	12
2.1 Résumé des contraintes et des demandes	12
2.2 Mise en place	12
2.2.1 Sur VMWare	12
2.2.2 Sur les Machines	12
2.2.2.1 Rôle et état des ports	13
2.3 Test STP	14
2.4 Configuration de RSTP	15
2.5 Test RSTP	15
3 tolérance de panne, couche "access"	17
4 Redondance de Routeurs	18

PARTAGE RÉSEAU NFS

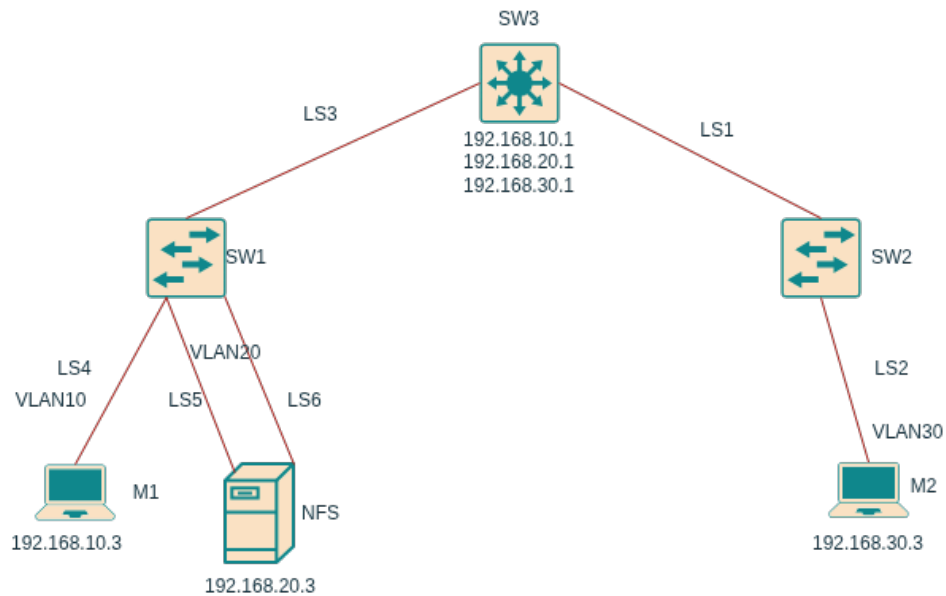


Figure 1.1: Maquette du réseau.

1.1 RÉSUMÉ DES CONTRAINTES ET DES DEMANDES:

- Les Switchs 1 et 2 n'utilisent que des fonctions de niveau 2.
- Le Switch 3 peut utiliser des fonctions de niveau 3.
- Les machines 1 et 2 doivent être isolées.
- Les machines 1 et 2 doivent avoir accès au serveur NFS.
- Les machines 1 et 2 doivent, pour transmettre via NFS au serveur, uniquement dans leurs dossiers respectifs.
- Tolérance à la panne sur les interfaces du serveur NFS.

Plan :

- Pour assurer l'isolation des machines, elles seront placées dans des VLANs différents et la communication entre ces réseaux sera interdite par des règles de pare-feu sur le Switch-3, qui agira comme RPD pour les deux machines. On peut également placer des règles pour restreindre les types de communications entre les machines et le serveur NFS (couper SSH, http, https, ...).

- Pour garantir la disponibilité des liens sur le serveur NFS, deux interfaces seront configurées en LACP mode [actif-backup](#) et connectées au Switch-1. En cas de défaillance du lien actif, le second lien prendra le relais.
- Pour le serveur NFS, nfs-kernel-server devra être configuré pour créer des partages réseau et définir les modalités de montage pour les clients.

1.2 MISE EN PLACE

On a donc besoin de mettre en place 3 VLANs, un pare-feu, une agrégation de liens et un serveur NFS.

1.2.1 SUR VMWARE

M1

```
adapter 1 : host only
adapter 2 : lan segment 4
```

M2

```
adapter 1 : host only
adapter 2 : lan segment 2
```

NFS

```
adapter 1 : host only
adapter 2 : lan segment 5
adapter 3 : lan segment 6
```

SW1

```
adapter 1 : host only
adapter 2 : lan segment 3
adapter 3 : lan segment 4
adapter 4 : lan segment 5
adapter 5 : lan segment 6
```

SW2

```
adapter 1 : host only
adapter 2 : lan segment 1
adapter 3 : lan segment 2
```

SW3

```
adapter 1 : host only
adapter 2 : lan segment 1
adapter 3 : lan segment 3
```

Toutes les machines hors switch sont des VM Debian 13 n'ayant pas de Display Manager installé. Chacune avec 2 coeurs et 677 MiB de RAM.



Figure 1.2: [screenfetch](#)

1.2.2 SUR LES MACHINES :

Configuration des VLAN :

Switch 1

```
net add hostname sw1
net add bridge bridge ports swp1,swp2,swp3,swp4
net add bridge bridge vlan-aware
net add interface swp2 bridge access 10
net add interface swp3,swp4 bridge access 20
net add interface swp1 bridge trunk vlans 10,20
net commit
```

```
cumulus@sw1:mgmt:~$ net show interface
```

State	Name	Spd	MTU	Mode	LLDP	Summary
UP	lo	N/A	65536	Loopback		IP: 127.0.0.1/8
	lo					IP: ::1/128
UP	eth0	10G	1500	Mgmt	sw2 (eth0)	Master: mgmt(UP)
	eth0					IP: 192.168.85.142/24(DHCP)
UP	swp1	1G	9216	Trunk/L2	sw3 (swp2)	Master: bridge(UP)
UP	swp2	1G	9216	Access/L2		Master: bridge(UP)
UP	swp3	1G	9216	Access/L2		Master: bridge(UP)
UP	swp4	1G	9216	Access/L2		Master: bridge(UP)
UP	bridge	N/A	9216	Bridge/L2		
UP	mgmt	N/A	65536	VRF		IP: 127.0.0.1/8
	mgmt					IP: ::1/128

```
cumulus@sw1:mgmt:~$ net show bridge vlan
```

Interface	VLAN	Flags
swp1	1	PVID, Egress Untagged
	10	
	20	
swp2	10	PVID, Egress Untagged
swp3	20	PVID, Egress Untagged
swp4	20	PVID, Egress Untagged

Switch 2

```
net add hostname sw2
net add bridge bridge ports swp1,swp2
net add bridge bridge vlan-aware
net add interface swp2 bridge access 30
net add interface swp1 bridge trunk vlans 20,30
net commit
```

```
cumulus@sw2:mgmt:~$ net show interface
```

State	Name	Spd	MTU	Mode	LLDP	Summary
UP	lo	N/A	65536	Loopback		IP: 127.0.0.1/8
	lo					IP: ::1/128
UP	eth0	10G	1500	Mgmt	sw1 (eth0)	Master: mgmt(UP)
	eth0					IP: 192.168.85.141/24(DHCP)
UP	swp1	1G	9216	Trunk/L2	sw3 (swp1)	Master: bridge(UP)
UP	swp2	1G	9216	Access/L2		Master: bridge(UP)
UP	bridge	N/A	9216	Bridge/L2		

```
UP      mgmt      N/A  65536  VRF
        mgmt
IP: 127.0.0.1/8
IP: ::1/128
```

```
cumulus@sw2:mgmt:~$ net show bridge vlan
```

```
Interface  VLAN  Flags
-----
swp1       1    PVID, Egress Untagged
           20
           30
swp2       30    PVID, Egress Untagged
```

Switch 3

```
net add hostname sw3
```

```
net add bridge bridge ports swp1,swp2
net add bridge bridge vlan-aware
```

```
net add interface swp2 bridge trunk vlans 10,20
net add interface swp1 bridge trunk vlans 20,30
```

```
net add vlan 10 ip address 192.168.10.1/24
net add vlan 20 ip address 192.168.20.1/24
net add vlan 30 ip address 192.168.30.1/24
net commit
```

```
cumulus@sw3:mgmt:~$ net show interface
```

State	Name	Spd	MTU	Mode	LLDP	Summary
UP	lo	N/A	65536	Loopback		IP: 127.0.0.1/8 IP: ::1/128
UP	eth0	10G	1500	Mgmt	sw1 (eth0)	Master: mgmt(UP) IP: 192.168.85.140/24(DHCP)
UP	swp1	1G	9216	Trunk/L2	sw2 (swp1)	Master: bridge(UP)
UP	swp2	1G	9216	Trunk/L2	sw1 (swp1)	Master: bridge(UP)
UP	bridge	N/A	9216	Bridge/L2		
UP	mgmt	N/A	65536	VRF		IP: 127.0.0.1/8 IP: ::1/128
UP	vlan10	N/A	9216	Interface/L3		IP: 192.168.10.1/24
UP	vlan20	N/A	9216	Interface/L3		IP: 192.168.20.1/24
UP	vlan30	N/A	9216	Interface/L3		IP: 192.168.30.1/24

```
cumulus@sw3:mgmt:~$ net show bridge vlan
```

```
Interface  VLAN  Flags
-----
swp1       1    PVID, Egress Untagged
           20
           30
swp2       1    PVID, Egress Untagged
           10
           20
bridge     10
           20
           30
```

1.2.3 EXPLICATION DES COMMANDES

- `net add hostname sw1`
 - Définit le nom d'hôte du switch.
- `net add bridge bridge ports swp1,swp2,swp3,swp4`
 - Ajoute les interfaces physiques (swp1 à swp4) au bridge principal.
- `net add bridge bridge vlan-aware`
 - Active le mode "VLAN-aware" du bridge pour permettre la gestion des VLANs.
- `net add interface swp2 bridge access 10`
`net add interface swp3,swp4 bridge access 20`
 - Configure les interfaces en mode accès:
 - swp2 appartient au VLAN 10.
 - swp3 et swp4 appartiennent au VLAN 20.
- `net add interface swp1 bridge trunk vlans 10,20`
 - Configure swp1 en mode trunk pour transporter les VLANs 10 et 20.
- `net add vlan 10 ip address 192.168.10.1/24`
`net add vlan 20 ip address 192.168.20.1/24`
`net add vlan 30 ip address 192.168.30.1/24`
 - Crée les interfaces VLAN et assigne les adresses IP de gestion:
 - VLAN 10: 192.168.10.0/24
 - VLAN 20: 192.168.20.0/24
 - VLAN 30: 192.168.30.0/24
- `net commit`
 - Valide et applique toutes les modifications précédentes, analogue à un commit sur Git.

1.3 CONFIGURATION DES ADRESSES IP SUR LES MACHINES

1.3.1 MACHINE 1

Dans le fichier

```
/etc/network/interfaces :  
auto ens36  
iface ens36 inet static  
    address 192.168.10.3/24  
    gateway 192.168.10.1
```

```

root@debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:3b:fa:96 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx00c293bfa96
    inet 192.168.59.146/24 brd 192.168.59.255 scope global dynamic noprefixroute ens33
        valid_lft 1683sec preferred_lft 1458sec
    inet6 fe80::87ea:ca07:2889:30ea/64 scope link
        valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:3b:fa:a0 brd ff:ff:ff:ff:ff:ff
    altname enp2s4
    altname enx00c293bfaa0
    inet 192.168.10.3/24 brd 192.168.10.255 scope global ens36
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe3b:faa0/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@debian:~#

```

Figure 1.3: Configuration IP de la Machine 1.

1.3.2 MACHINE 2

```

auto ens36
iface ens36 inet static
    address 192.168.30.3/24
    gateway 192.168.30.1

```

```

root@debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:db:e3:a3 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx00c29dbe3a3
    inet 192.168.59.145/24 brd 192.168.59.255 scope global dynamic noprefixroute ens33
        valid_lft 1687sec preferred_lft 1462sec
    inet6 fe80::6a89:ec8e:78c2:74c8/64 scope link
        valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:db:e3:ad brd ff:ff:ff:ff:ff:ff
    altname enp2s4
    altname enx00c29dbe3ad
    inet 192.168.30.3/24 brd 192.168.30.255 scope global ens36
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fedb:e3ad/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@debian:~# _

```

Figure 1.4: Configuration IP de la Machine 2.

1.3.3 SERVEUR NFS

```

auto bond0
iface bond0 inet static
    address 192.168.20.3/24
    gateway 192.168.20.1
    bond-slaves ens36 ens37
    bond-mode active-backup
    bond-miimon 100
    bond-primary eth1

```

```

auto ens36
iface ens36 inet manual
    bond-master bond0

```

```

auto ens37

```

```
iface ens37 inet manual
    bond-master bond0
```

1.3.3.1 EXPLICATION DES PARAMÈTRES DU BOND

- Définir une adresse.
- Définir une passerelle par défaut (les trois machines ont comme passerelle par défaut le Switch-3).
- Indiquer les interfaces membres de l'agrégation.
- Indiquer le mode d'agrégation.
- Indiquer le délai de bascule.
- Indiquer l'interface principale (celle qui serait UP par défaut).
- Indiquer le maître d'agrégation aux interfaces.

```
root@debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:ab:a8:67 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx00c29aba867
    inet 192.168.59.147/24 brd 192.168.59.255 scope global dynamic noprefixroute ens33
        valid_lft 1720sec preferred_lft 1495sec
    inet6 fe80::4a00:1534:c0dc:c5a0/64 scope link
        valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,SLAVE,UP,LOWER_UP> mtu 1500 qdisc fq_codel master bond0 state UP group default qlen 1000
    link/ether 00:0c:29:ab:a8:71 brd ff:ff:ff:ff:ff:ff
    altname enp2s4
    altname enx00c29aba871
    inet6 fe80::ba1c:7a59:d06e:84f3/64 scope link tentative noprefixroute
        valid_lft forever preferred_lft forever
4: ens37: <BROADCAST,MULTICAST,SLAVE,UP,LOWER_UP> mtu 1500 qdisc fq_codel master bond0 state UP group default qlen 1000
    link/ether 00:0c:29:ab:a8:71 brd ff:ff:ff:ff:ff:ff permaddr 00:0c:29:ab:a8:7b
    altname enp2s5
    altname enx00c29aba87b
    inet6 fe80::6151:7189:7316:a397/64 scope link tentative noprefixroute
        valid_lft forever preferred_lft forever
5: bond0: <BROADCAST,MULTICAST,MASTER,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 00:0c:29:ab:a8:71 brd ff:ff:ff:ff:ff:ff
    inet 192.168.20.3/24 brd 192.168.20.255 scope global bond0
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:23ff:feab:a671/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@debian:~#
```

Figure 1.5: Configuration IP du Serveur NFS.

On peut vérifier l'état du Bond : `cat /proc/net/bonding/bond0`

```

root@debian:~# cat /proc/net/bonding/bond0
Ethernet Channel Bonding Driver: v6.12.73+deb13-amd64

Bonding Mode: fault-tolerance (active-backup)
Primary Slave: None
Currently Active Slave: ens36
MII Status: up
MII Polling Interval (ms): 100
Up Delay (ms): 0
Down Delay (ms): 0
Peer Notification Delay (ms): 0

Slave Interface: ens36
MII Status: up
Speed: 1000 Mbps
Duplex: full
Link Failure Count: 0
Permanent HW addr: 00:0c:29:ab:a8:71
Slave queue ID: 0

Slave Interface: ens37
MII Status: up
Speed: 1000 Mbps
Duplex: full
Link Failure Count: 0
Permanent HW addr: 00:0c:29:ab:a8:7b
Slave queue ID: 0
root@debian:~# █

```

Figure 1.6: Vérification de l'état du lien d'agrégation (bond).

1.4 CONFIGURATION DU PARE-FEU

1.4.1 SUR SWITCH-3

Dans `/etc/cumulus/acl/policy.d/50_custom.rules`
[iptables]

```

-A FORWARD -s 192.168.30.3 -d 192.168.10.3 -j DROP
-A FORWARD -s 192.168.10.3 -d 192.168.30.3 -j DROP

```

1.4.1.1 EXPLICATION DES RÈGLES DE PARE-FEU

Ces règles de pare-feu ont pour but de détruire tout paquet entre les deux réseaux afin d'assurer leur isolation.

On peut tester que les machines ne peuvent pas se pinguer :

```

3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:db:e3:ad brd ff:ff:ff:ff:ff:ff
    altname enp2s4
    altname enx000c29dbe3ad
    inet 192.168.30.3/24 brd 192.168.30.255 scope global ens36
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fedb:e3ad/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@debian:~# ping 192.168.20.3
PING 192.168.20.3 (192.168.20.3) 56(84) bytes of data.
64 bytes from 192.168.20.3: icmp_seq=1 ttl=63 time=1.08 ms
64 bytes from 192.168.20.3: icmp_seq=2 ttl=63 time=2.14 ms
64 bytes from 192.168.20.3: icmp_seq=3 ttl=63 time=2.53 ms
64 bytes from 192.168.20.3: icmp_seq=4 ttl=63 time=1.03 ms
^C
--- 192.168.20.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.026/1.695/2.529/0.654 ms
root@debian:~# ping 192.168.10.3
PING 192.168.10.3 (192.168.10.3) 56(84) bytes of data.
^C
--- 192.168.10.3 ping statistics ---
19 packets transmitted, 0 received, 100% packet loss, time 18427ms

root@debian:~# _

```

Figure 1.7: Test de communication entre les machines.

On peut voir que la Machine-2 peut communiquer avec le serveur NFS mais pas avec la Machine-1

1.5 CONFIGURATION DU PARTAGE NFS

1.5.1 SUR LE SERVEUR NFS

```
apt install nfs-kernel-server
```

```
mkdir /machine1
```

```
mkdir /machine2
```

```
chown nobody:nogroup /machine1 /machine2
```

```
chmod 777 /machine1 /machine2
```

On ajoute dans /etc/exports :

```
/machine1 192.168.10.0/24(rw,sync,no_subtree_check)
```

```
/machine2 192.168.30.0/24(rw,sync,no_subtree_check)
```

Ensuite :

```
exportfs -a
```

```
systemctl restart nfs-kernel-server
```

```
# Machine-1
```

```
mkdir /mnt/backup_m1
```

```
mount -t nfs -o vers=4 192.168.20.3:/machine1 /mnt/backup_m1
```

1.5.1.1 TEST DU TRANSFERT NFS

On peut ensuite tester que le transfert fonctionne en créant un fichier dans /mnt/backup_m1 et vérifier qu'il apparaît bien sur le serveur NFS.

```
# Machine-1
```

```
touch /mnt/backup_m1/test.ms
```

```
# Serveur NFS
```

```
ls -la /machine1
```

```

root@debian:~# touch /mnt/backup_m1/test.ms
root@debian:~# ls /mnt/backup_m1/
NAME      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda       8:0    0   200  0 disk
|-sda1    8:1    0  19.2G  0 part /
|-sda2    8:2    0    1K  0 part
|-sda5    8:5    0   768M  0 part [SWAP]
sr0       1:0    1   754M  0 rom
root@debian:~# touch test.ms /mnt/backup_m1/
root@debian:~# touch /mnt/backup_m1/test.ms
root@debian:~# _

~
~
~
/etc/exports
root@debian:~# ls -la /machine1
total 0
drwxr-xr-x 2 nobody nogroup 4096 Mar  9 21:32 .
drwxr-xr-x 20 root    root    4096 Mar  9 21:26 ..
-rw-r--r-- 1 nobody nogroup   0 Mar  9 21:32 test.ms
root@debian:~#

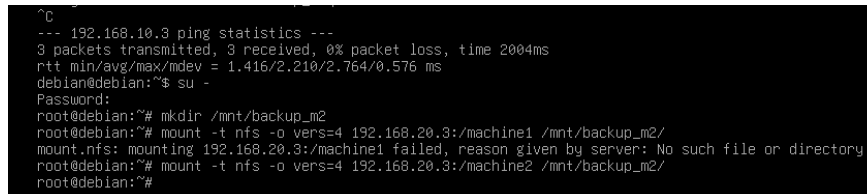
```

Figure 1.8: En haut, la création du fichier test.ms dans le dossier /mnt/backup_m1 sur la Machine-1. En bas, on constate la présence de test.ms dans /machine1 sur le Serveur-NFS.

1.5.1.2 TEST D'ACCÈS RESTREINT

On peut aussi vérifier que l'on ne peut pas monter le partage (share) si l'on ne fait pas partie du réseau désigné :

```
# Machine-2
mkdir /mnt/backup_m2
mount -t nfs -o vers=4 192.168.20.3:/machine1 /mnt/backup_m2
```

A terminal window showing the execution of commands on Machine-2. It starts with a ping to 192.168.10.3, then switches to root using 'su -'. The user runs 'mkdir /mnt/backup_m2'. Then, they attempt to mount the NFS share '192.168.20.3:/machine1' to '/mnt/backup_m2' using 'mount -t nfs -o vers=4'. The command fails with the error 'mount.nfs: mounting 192.168.20.3:/machine1 failed, reason given by server: No such file or directory'. A second attempt to mount '192.168.20.3:/machine2' also fails with the same error message.

```
^C
--- 192.168.10.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.416/2.210/2.764/0.576 ms
debian@debian:~$ su -
Password:
root@debian:~# mkdir /mnt/backup_m2
root@debian:~# mount -t nfs -o vers=4 192.168.20.3:/machine1 /mnt/backup_m2/
mount.nfs: mounting 192.168.20.3:/machine1 failed, reason given by server: No such file or directory
root@debian:~# mount -t nfs -o vers=4 192.168.20.3:/machine2 /mnt/backup_m2/
root@debian:~#
```

Figure 1.9: Échec de la tentative de montage d'un partage non autorisé.

Comme attendu, la Machine-2 ne peut se connecter qu'à son partage et pas à celui dédié à la Machine-1.

REDONDANCE DE LIENS

2.1 RÉSUMÉ DES CONTRAINTES ET DES DEMANDES

- Les trois commutateurs (Switch 1, 2 et 3) n'utilisent que des fonctions de niveau 2.
- Il est impératif de mettre en place un réseau redondant entre les trois commutateurs tout en garantissant l'absence de boucle L2.
- La solution doit permettre une reprise du trafic la plus rapide possible en cas de coupure d'un lien.

Plan :

- Nous allons configurer les connexions physiques et logiques (ponts) sur les trois commutateurs pour lier les machines.
- Afin d'éviter les tempêtes de diffusion (boucles L2) causées par la topologie en anneau, nous allons forcer l'utilisation du protocole STP (Spanning Tree Protocol) standard, analyser les rôles des ports, puis simuler une panne de lien pour observer le temps de convergence.
- Ensuite, nous passerons sur le protocole RSTP (Rapid Spanning Tree Protocol) pour comparer les performances de basculement et répondre à l'exigence de reprise rapide du trafic.

2.2 MISE EN PLACE

2.2.1 SUR VMWARE

- | | |
|--|--|
| • Machine 1 : lan segment 1 (IP : 192.168.0.1) | • Switch 1 : lan segment 1, lan segment 4, lan segment 6 |
| • Machine 2 : lan segment 2 (IP : 192.168.0.2) | • Switch 2 : lan segment 2, lan segment 4, lan segment 5 |
| • Machine 3 : lan segment 3 (IP : 192.168.0.3) | • Switch 3 : lan segment 3, lan segment 5, lan segment 6 |

2.2.2 SUR LES MACHINES

On peut forcer l'activation du protocole avec :

```
# Sur chaque switch
```

```
sudo mstctl setforcevers bridge stp
```

- `mstctl` est la commande qui sert à configurer mstpd (Multiple Spanning Tree daemon)
- `setforcevers` sert à spécifier une version du protocole, ici stp (default mstp)¹

¹[mstctl.8 | sourcecode - mstctl](#)

NB: Le protocole STP est activé par défaut sur Cumulus

2.2.2.1 RÔLE ET ÉTAT DES PORTS

On peut voir l'état du protocole avec la commande `net show bridge spanning-tree` :

```
cumulus@cumulus:mgmt:~$ net show bridge spanning-tree
Bridge info
enabled          yes
bridge id        8.000.00:0C:29:16:76:00
Priority:         32768
Address:         00:0C:29:16:76:00
This bridge is root.

designated root 8.000.00:0C:29:16:76:00
Priority:         32768
Address:         00:0C:29:16:76:00

root port        none
path cost        0          internal path cost  0
max age          20          bridge max age      20
forward delay    15          bridge forward delay 15
tx hold count    6           max hops            20
hello time       2           ageing time          300
force protocol version stp

INTERFACE  STATE  ROLE  EDGE
-----
swp1       forw  Desg
swp2       forw  Desg
swp3       forw  Desg
cumulus@cumulus:mgmt:~$

cumulus@cumulus:mgmt:~$ net show bridge spanning-tree
Bridge info
enabled          yes
bridge id        8.000.00:0C:29:16:76:00
Priority:         32768
Address:         00:0C:29:16:76:00
This bridge is root.

designated root 8.000.00:0C:29:16:76:00
Priority:         32768
Address:         00:0C:29:16:76:00

root port        swp2 (#2)
path cost        20000      internal path cost  0
max age          20          bridge max age      20
forward delay    15          bridge forward delay 15
tx hold count    6           max hops            20
hello time       2           ageing time          300
force protocol version stp

INTERFACE  STATE  ROLE  EDGE
-----
swp1       forw  Desg
swp2       forw  Root
swp3       forw  Desg
cumulus@cumulus:mgmt:~$

cumulus@cumulus:mgmt:~$ net show bridge spanning-tree
Bridge info
enabled          yes
bridge id        8.000.00:0C:29:BD:54:5F
Priority:         32768
Address:         00:0C:29:BD:54:5F
designated root 8.000.00:0C:29:16:76:00
Priority:         32768
Address:         00:0C:29:16:76:00

root port        swp2 (#2)
path cost        20000      internal path cost  0
max age          20          bridge max age      20
forward delay    15          bridge forward delay 15
tx hold count    6           max hops            20
hello time       2           ageing time          300
force protocol version stp

INTERFACE  STATE  ROLE  EDGE
-----
swp1       forw  Desg
swp2       forw  Root
swp3       disc  Altn
cumulus@cumulus:mgmt:~$
```

Figure 2.1: Switch 1, 2 et 3, respectivement

- Switch 1 : Son port swp2 est le Root Port (Root) (état Forwarding) avec un coût de 20000. Son port swp1 est Designated (Desg) (état Forwarding).
Prévention de la boucle : Le port swp3 du Switch 1 a été placé en rôle Alternate (Altn) et en état Discarding (disc). Il bloque le trafic pour casser la boucle L2, tout en restant à l'écoute d'éventuels changements de topologie
- Switch 2 (Root Bridge) : Ce commutateur a été élu pont racine (This bridge is root) car il possède l'adresse MAC la plus faible parmi ceux ayant la priorité par défaut (32768). Ses trois ports (swp1, swp2, swp3) sont en rôle Designated (Desg) et en état Forwarding (forw).
- Switch 3 : Son port swp2 a été élu Root Port (Root) (état Forwarding) car c'est le chemin le plus court vers le Root Bridge. Les autres ports (swp1, swp3) sont en rôle Designated (Desg) (état Forwarding).

2.3 TEST STP

Pour simuler une panne, on va desactiver un des lien actif. Un des lien passif devrait alors prendre le relais pour assurer la communication.

On coupe le lien avec la commande `sudo ip link set swp2 down`: Un ping est lancé sur une des machines.

```

root@debian:~# ping 192.168.0.2
PING 192.168.0.2 (192.168.0.2) 56(84) bytes of data.
64 bytes from 192.168.0.2: icmp_seq=1 ttl=64 time=5.83 ms
64 bytes from 192.168.0.2: icmp_seq=2 ttl=64 time=2.93 ms
64 bytes from 192.168.0.2: icmp_seq=3 ttl=64 time=2.90 ms
64 bytes from 192.168.0.2: icmp_seq=4 ttl=64 time=3.47 ms
64 bytes from 192.168.0.2: icmp_seq=5 ttl=64 time=3.14 ms
64 bytes from 192.168.0.2: icmp_seq=6 ttl=64 time=2.96 ms
64 bytes from 192.168.0.2: icmp_seq=7 ttl=64 time=2.92 ms
64 bytes from 192.168.0.2: icmp_seq=8 ttl=64 time=4.57 ms
64 bytes from 192.168.0.2: icmp_seq=9 ttl=64 time=3.29 ms
64 bytes from 192.168.0.2: icmp_seq=10 ttl=64 time=2.83 ms
64 bytes from 192.168.0.2: icmp_seq=40 ttl=64 time=6.46 ms
64 bytes from 192.168.0.2: icmp_seq=41 ttl=64 time=3.92 ms
64 bytes from 192.168.0.2: icmp_seq=42 ttl=64 time=3.57 ms
64 bytes from 192.168.0.2: icmp_seq=43 ttl=64 time=3.57 ms
64 bytes from 192.168.0.2: icmp_seq=44 ttl=64 time=3.79 ms
64 bytes from 192.168.0.2: icmp_seq=45 ttl=64 time=3.90 ms
64 bytes from 192.168.0.2: icmp_seq=46 ttl=64 time=4.83 ms
64 bytes from 192.168.0.2: icmp_seq=47 ttl=64 time=4.11 ms
64 bytes from 192.168.0.2: icmp_seq=48 ttl=64 time=3.76 ms
64 bytes from 192.168.0.2: icmp_seq=49 ttl=64 time=3.96 ms
64 bytes from 192.168.0.2: icmp_seq=50 ttl=64 time=3.62 ms
64 bytes from 192.168.0.2: icmp_seq=51 ttl=64 time=4.01 ms
^C
--- 192.168.0.2 ping statistics ---
51 packets transmitted, 22 received, 56.8627% packet loss, time 50748ms
rtt min/avg/max/mdev = 2.830/3.833/6.457/0.902 ms
root@debian:~# _

```

Figure 2.2: Ping vers machine 2 lancé depuis machine 1

```

root@debian:~# sudo tcpdump -i ens3 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:04:43.393747 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 1, length 64
18:04:43.397559 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 1, length 64
18:04:44.396290 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 2, length 64
18:04:44.399369 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 2, length 64
18:04:45.398029 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 3, length 64
18:04:45.401557 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 3, length 64
18:04:46.400300 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 4, length 64
18:04:46.403448 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 4, length 64
18:04:47.402849 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 5, length 64
18:04:47.406275 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 5, length 64
18:04:48.405013 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 6, length 64
18:04:48.408114 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 6, length 64
18:04:49.406752 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 7, length 64
18:04:49.409651 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 7, length 64
18:04:50.408961 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 8, length 64
18:04:50.413516 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 8, length 64
18:04:51.411627 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 9, length 64
18:04:51.416874 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 9, length 64
18:04:52.413392 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 10, length 64
18:04:52.417385 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 10, length 64
18:04:53.416075 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 11, length 64
18:04:53.420266 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 11, length 64
18:04:54.418193 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 12, length 64
18:04:54.422321 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 12, length 64
18:04:55.420810 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 13, length 64
18:04:55.424572 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 13, length 64
18:04:56.423247 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 14, length 64
18:04:56.427011 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 14, length 64
18:04:57.425898 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 15, length 64
18:04:57.429799 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 15, length 64
18:04:58.428237 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 16, length 64
18:04:58.431871 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 16, length 64
18:04:59.430314 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 17, length 64
17:57:30.052477 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 25, length 64
17:57:31.076497 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 26, length 64
17:57:32.100361 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 27, length 64
17:57:33.124613 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 28, length 64
17:57:34.149033 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 29, length 64
17:57:35.172846 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 30, length 64
17:57:36.197055 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 31, length 64
17:57:37.220247 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 32, length 64
17:57:38.244985 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 33, length 64
17:57:39.268376 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 34, length 64
17:57:40.292765 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 35, length 64
17:57:41.316247 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 36, length 64
17:57:42.340711 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 37, length 64
17:57:43.364555 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 38, length 64
17:57:44.388339 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 39, length 64
17:57:45.412244 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 40, length 64
17:57:45.418639 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 1, seq 40, length 64
17:57:46.414702 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 41, length 64
17:57:46.418583 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 1, seq 41, length 64
17:57:47.417496 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 42, length 64
17:57:47.421015 IP 192.168.0.2 > 192.168.0.1: ICMP echo request, id 1, seq 42, length 64
17:57:48.420208 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 43, length 64
17:57:48.423665 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 1, seq 43, length 64
17:57:49.421915 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 44, length 64
17:57:49.425666 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 1, seq 44, length 64

```

Figure 2.3: Tcpcdump sur une des machines

Observations : Le flux ICMP a été interrompu à la séquence `icmp_seq=10`. Le trafic n’a repris qu’à la séquence `icmp_seq=40`. On observe une perte : 51 packets transmitted, 22 received, 56.8627% packet loss, time 50748ms

Le lien passif a bien prit le relais mais avec un délais qui a permit une perte de paquets.

Explication : Avec le protocole STP classique (802.1D), lorsqu’un lien tombe, le réseau doit recalculer la topologie. Le port bloqué (swp3) doit passer par plusieurs états intermédiaires temporisés (Listening et Learning) avant de passer en Forwarding. Les timers par défaut (max age 20, forward delay 15) imposent un délai de convergence, ce qui explique les 30 requêtes ICMP perdues (de la séquence 11 à 39) avant la reprise de la connectivité.

En théorie, le délai est

$$2 * \text{forward delay} = 2 * 15s = 30s \quad (2.1)$$

(15s en *Listening*, puis 15s en *Learning*), ce qui coïncide avec les 30 paquets ICMP perdu, qui sont envoyés à une fréquence de (1 paquet) s^{-1} .

2.4 CONFIGURATION DE RSTP

Pour répondre à la contrainte de “reprise du trafic la plus rapide possible”, nous modifions la version du protocole sur les commutateurs pour utiliser RSTP (802.1w).

Pour cela, on force à nouveau un protocole spécifique : `sudo mstpcctl setforcevers bridge rstp`.

2.5 TEST RSTP

On coupe à nouveau le lien (`sudo ip link set swp2 down`)

```

64 bytes from 192.168.0.2: icmp_seq=23 ttl=64 time=3.55 ms
64 bytes from 192.168.0.2: icmp_seq=24 ttl=64 time=3.52 ms
64 bytes from 192.168.0.2: icmp_seq=25 ttl=64 time=3.61 ms
64 bytes from 192.168.0.2: icmp_seq=26 ttl=64 time=3.81 ms
64 bytes from 192.168.0.2: icmp_seq=27 ttl=64 time=3.83 ms
64 bytes from 192.168.0.2: icmp_seq=28 ttl=64 time=3.48 ms
64 bytes from 192.168.0.2: icmp_seq=29 ttl=64 time=3.93 ms
64 bytes from 192.168.0.2: icmp_seq=30 ttl=64 time=3.66 ms
64 bytes from 192.168.0.2: icmp_seq=31 ttl=64 time=3.59 ms
64 bytes from 192.168.0.2: icmp_seq=32 ttl=64 time=4.20 ms
64 bytes from 192.168.0.2: icmp_seq=33 ttl=64 time=3.75 ms
64 bytes from 192.168.0.2: icmp_seq=34 ttl=64 time=3.76 ms
64 bytes from 192.168.0.2: icmp_seq=35 ttl=64 time=3.79 ms
64 bytes from 192.168.0.2: icmp_seq=36 ttl=64 time=4.59 ms
64 bytes from 192.168.0.2: icmp_seq=37 ttl=64 time=3.88 ms
64 bytes from 192.168.0.2: icmp_seq=38 ttl=64 time=3.64 ms
64 bytes from 192.168.0.2: icmp_seq=39 ttl=64 time=3.59 ms
64 bytes from 192.168.0.2: icmp_seq=40 ttl=64 time=3.93 ms
64 bytes from 192.168.0.2: icmp_seq=41 ttl=64 time=3.99 ms
64 bytes from 192.168.0.2: icmp_seq=42 ttl=64 time=3.47 ms
64 bytes from 192.168.0.2: icmp_seq=43 ttl=64 time=3.78 ms
64 bytes from 192.168.0.2: icmp_seq=44 ttl=64 time=4.18 ms
64 bytes from 192.168.0.2: icmp_seq=45 ttl=64 time=3.34 ms
64 bytes from 192.168.0.2: icmp_seq=46 ttl=64 time=3.37 ms
64 bytes from 192.168.0.2: icmp_seq=47 ttl=64 time=5.74 ms
64 bytes from 192.168.0.2: icmp_seq=48 ttl=64 time=4.42 ms
64 bytes from 192.168.0.2: icmp_seq=49 ttl=64 time=3.73 ms
64 bytes from 192.168.0.2: icmp_seq=50 ttl=64 time=3.67 ms
64 bytes from 192.168.0.2: icmp_seq=51 ttl=64 time=5.83 ms
64 bytes from 192.168.0.2: icmp_seq=52 ttl=64 time=3.47 ms
64 bytes from 192.168.0.2: icmp_seq=53 ttl=64 time=3.88 ms
64 bytes from 192.168.0.2: icmp_seq=54 ttl=64 time=4.25 ms
64 bytes from 192.168.0.2: icmp_seq=55 ttl=64 time=3.51 ms
64 bytes from 192.168.0.2: icmp_seq=56 ttl=64 time=3.51 ms
64 bytes from 192.168.0.2: icmp_seq=57 ttl=64 time=3.73 ms
64 bytes from 192.168.0.2: icmp_seq=58 ttl=64 time=3.92 ms
64 bytes from 192.168.0.2: icmp_seq=59 ttl=64 time=3.70 ms
64 bytes from 192.168.0.2: icmp_seq=60 ttl=64 time=4.19 ms
64 bytes from 192.168.0.2: icmp_seq=61 ttl=64 time=3.31 ms
64 bytes from 192.168.0.2: icmp_seq=62 ttl=64 time=4.02 ms
64 bytes from 192.168.0.2: icmp_seq=63 ttl=64 time=3.36 ms
64 bytes from 192.168.0.2: icmp_seq=64 ttl=64 time=4.04 ms
64 bytes from 192.168.0.2: icmp_seq=65 ttl=64 time=3.57 ms
64 bytes from 192.168.0.2: icmp_seq=66 ttl=64 time=4.09 ms
64 bytes from 192.168.0.2: icmp_seq=67 ttl=64 time=3.94 ms
^C
--- 192.168.0.2 ping statistics ---
67 packets transmitted, 67 received, 0% packet loss, time 66153ms
rtt min/avg/max/mdev = 2.965/3.860/5.735/0.509 ms
root@debian: #

```

```

root@debian:~# sudo tcpdump -i ens33 icmp
tcpdump: verbose output suppressed, use -v(v)... for full protocol decode
listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:04:43.397747 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 1, length 64
18:04:43.397559 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 1, length 64
18:04:44.396294 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 2, length 64
18:04:44.398950 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 2, length 64
18:04:45.398829 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 3, length 64
18:04:45.408357 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 3, length 64
18:04:46.408388 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 4, length 64
18:04:46.408348 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 4, length 64
18:04:47.402940 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 5, length 64
18:04:47.406275 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 5, length 64
18:04:48.408813 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 6, length 64
18:04:48.408314 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 6, length 64
18:04:49.406752 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 7, length 64
18:04:49.409653 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 7, length 64
18:04:50.408961 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 8, length 64
18:04:50.413516 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 8, length 64
18:04:51.411627 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 9, length 64
18:04:51.416874 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 9, length 64
18:04:52.413392 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 10, length 64
18:04:52.417380 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 10, length 64
18:04:53.416975 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 11, length 64
18:04:53.420260 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 11, length 64
18:04:54.418191 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 12, length 64
18:04:54.422321 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 12, length 64
18:04:55.428818 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 13, length 64
18:04:55.428972 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 13, length 64
18:04:56.423247 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 14, length 64
18:04:56.428011 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 14, length 64
18:04:57.425898 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 15, length 64
18:04:57.429779 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 15, length 64
18:04:58.428237 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 16, length 64
18:04:58.431871 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 16, length 64
18:04:59.430314 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 17, length 64

```

Observations : Contrairement au test précédent, l'interruption a été quasi imperceptible. Tcpdump et ping montrent une continuité quasi parfaite du trafic : 67 packets transmitted, 67 received, 0% packet loss, time 66153ms

Explication : Le RSTP réduit drastiquement le temps de convergence (généralement sous la seconde) car il n'utilise pas de timers fixes pour changer l'état des ports. Il intègre un mécanisme de négociation active (*Proposal/Agreement*) entre les commutateurs voisins. Ainsi, lorsque le lien principal (swp2) tombe sur le Switch 1, le port alternatif (swp3) passe immédiatement à l'état Forwarding sans devoir traverser les phases d'écoute et d'apprentissage, réduisant ainsi toute perte de paquets significative lors de notre test.²

²Understand Rapid Spanning Tree Protocol (802.1w)

TOLÉRANCE DE PANNE, COUCHE “ACCESS”

Redondance de Routeurs